

Importación y exportación de datos

IMPORTAR DATOS DESDE CUALQUIER TIPO DE ARCHIVO.....	2
EXPORTAR DATOS.....	4
EJEMPLOS DE IMPORTACIÓN.....	5
EJEMPLOS DE EXPORTACIÓN.....	11

IMPORTAR DATOS DESDE CUALQUIER TIPO DE ARCHIVO

Cuando trabajamos con datos, destinamos una importante cantidad de tiempo a la limpieza de los datos y para ello, una de las primeras acciones a realizar es la carga de los datos para conocerlos y ver a qué nos enfrentamos. Python mediante su librería Pandas nos ofrece la funcionalidad de importar y exportar datos mediante el uso de funciones sencillas de comprender y ejecutar.

Función	Descripción
<code>read_csv</code>	Crea dataframe con los datos que lee de un archivo tipo CSV (Comma-Separated Values). El delimitador por defecto es la coma.
<code>read_fwf</code>	Lee archivos de texto con columnas de ancho fijo, sin delimitadores
<code>read_clipboard</code>	Lee datos del portapapeles
<code>read_excel</code>	Lee datos tabulares de un archivo XLS o XLSX de Excel
<code>read_hdf</code>	Lee archivos HDF5 escritos por pandas
<code>read_html</code>	Lee todas las tablas que encuentra en el documento HTML dado
<code>read_json</code>	Lee datos desde un archivo, URL o cadena de texto JSON (Javascript Object Notation)
<code>read_parquet</code>	Lee el formato de archivo binario Parquet de Apace
<code>read_sql</code>	Lee los resultados de una consulta SQL (utilizando la biblioteca SQLAlchemy)
<code>read_sql_table</code>	Lee una tabla SQL completa (utilizando la biblioteca SQLAlchemy)
<code>read_xml</code>	Lee una tabla o datos de un archivo XML
<code>read_feather</code>	Lee el formato de archivo binario Feather
<code>read_spss</code>	Lee un archivo de datos creado por SPSS (.sav)
<code>read_stata</code>	Lee conjunto de datos de un formato de archivo Stata (.dta)

Estas funciones cuentan con argumentos opcionales que permiten personalizar la lectura de datos. Algunas de las opciones más comunes para leer archivos e importar datos con Python pandas son:

Argumento	Descripción
sep o delimiter	Caracteres o expresión regular que especifican el separador de campos en cada fila. Por ejemplo <code>sep="\t"</code> indica que están separados por tabuladores.
header	Especifica el número de fila que contiene los nombres de las columnas. Por defecto es cero, pero debería ser <code>None</code> si no hay fila de encabezado.
index_col	Especifica el número o nombres de columna que se debe usar como índice de fila en el resultado
skiprows	Lista de número de líneas que se deben ignorar al leer el archivo.
na_values	Secuencia de valores que se deben identificar como valores NA. Por ejemplo, puedes utilizar <code>na_values=["N/A", ""]</code> para indicar que los valores "N/A" y el texto vacío deben interpretarse como NA.
keep_default_na	Para usar la lista predeterminada de valores NA (not available) o no. Por defecto está a <code>True</code>
dtype	Especifica el tipo de datos de cada columna. Por ejemplo, <code>dtype={"col1": int, "col2": float}</code> indica que la columna "col1" contiene enteros y la columna "col2" contiene flotantes.
usecols	especifica una lista de nombres de columna que se deben leer del archivo. Por ejemplo, puedes utilizar <code>usecols=["col1", "col3"]</code> para leer solo las columnas "col1" y "col3" del archivo.
nrows	Número de filas que se leen desde el principio del archivo (sin contar el encabezado)
skip_footer	Número de líneas que se ignoran del final del archivo.
parse_dates	Especifica una lista de nombres de columnas que se deben interpretar como fechas <i>datetime</i> .
decimal	Especifica el separador de los decimales en números. Por defecto es "." pero puede ponerse como ",",
thousands	Especifica el separador de los miles para números. Por defecto es <code>None</code> pero puede ponerse como ",", o "."

Hay muchas otras opciones disponibles, y no todas las opciones están disponibles para todas las funciones de lectura de Pandas. Para más información de cada función de lectura recordad que podéis acudir a la [documentación online](#)

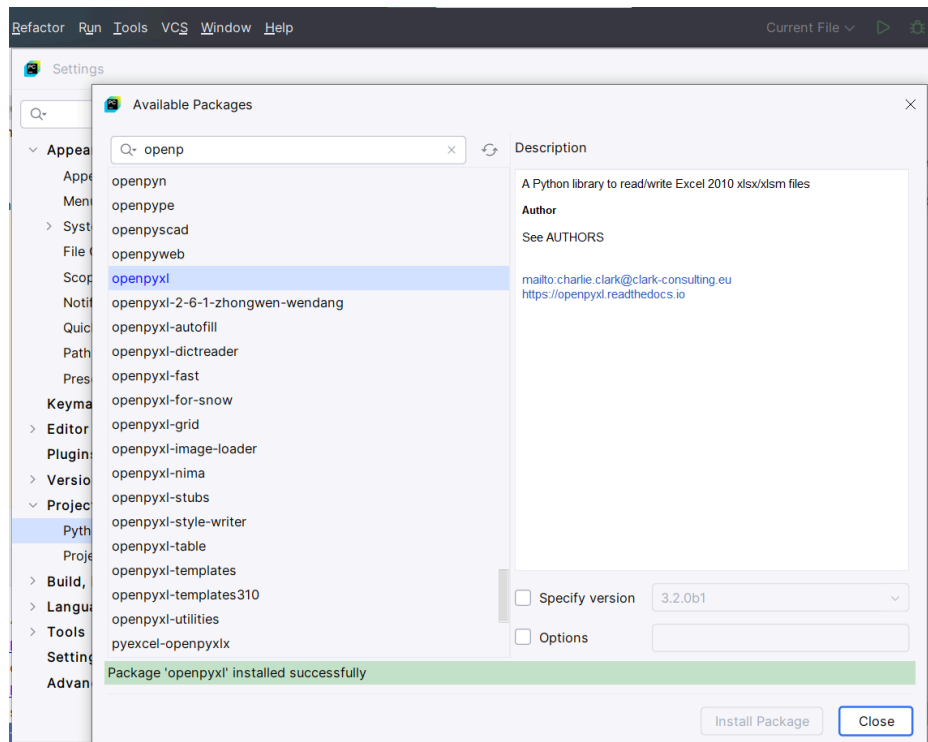
EXPORTAR DATOS

Los datos también se pueden exportar a un formato delimitado empleando los métodos `to_*` del objeto DataFrame.

Método	Descripción
<code>to_csv</code>	Escribe el objeto DataFrame en un archivo separado por comas (csv).
<code>to_clipboard</code>	Copia el objetos en el portapapeles del sistema
<code>to_excel</code>	Copia el objeto en un archivo de Excel
<code>to_html</code>	Representa el DataFrame como una tabla HTML
<code>to_json</code>	Convierte el objeto en un string JSON
<code>to_parquet</code>	Escribe el DataFrame en el formato Parquet
<code>to_sql</code>	Escribe los datos del DataFrame en una base de datos SQL
<code>to_xml</code>	Representa el DataFrame en un documento XML
<code>to_feather</code>	Escribe el DataFrame en formato feather
<code>to_stata</code>	Exporta el objeto DataFrame a formato Stata dta
<code>to_latex</code>	Convierte el objeto en una tabla LaTeX tabular, longtable o anidada

EJEMPLOS DE IMPORTACIÓN

Para algunas de las funciones, tenemos que instalar algunas dependencias que se irán indicando con los ejemplos. Para trabajar con archivos excel necesitamos añadir la librería openpyxl. En Pycharm le damos a File → Settings → Project: → Python Interpreter → +. En Jupyter con `pip install nombre_libreria`.



```
import pandas as pd
```

```
empleados = pd.read_excel("Lista-de-clientes.xlsx")  
print(empleados)
```

	ID	...	Grupo de clientes
0	C0001	...	A
1	C0002	...	E
2	C0003	...	E
3	C0004	...	E
4	C0005	...	D
..	...	...	...
994	C0995	...	B
995	C0996	...	D
996	C0997	...	A
997	C0998	...	D
998	C0999	...	A

```
[999 rows x 9 columns]
```

```
portapapeles = pd.read_clipboard()
print(portapapeles)
```

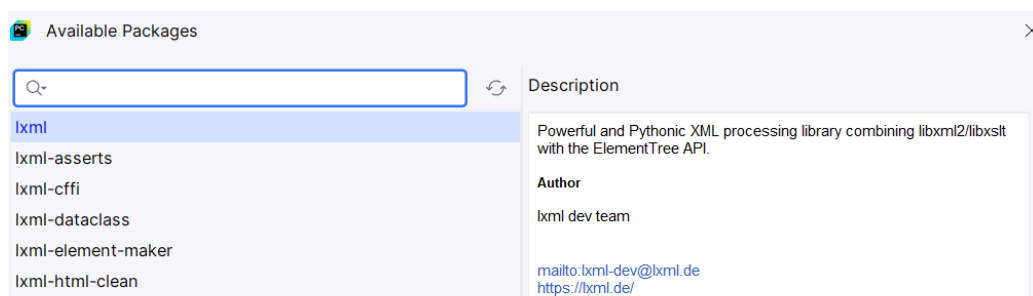
	ID	Nombre completo	Fecha de nacimiento
0	C0001	Leandra Anna Malo Alba	08/12/1984
1	C0002	Severo Granados Iglesia	12/08/1986
2	C0003	Lucho Andreu Amat	16/04/1990
3	C0004	Matias Mauricio Castillo Barrera	02/12/1996
4	C0005	Mauricio Guijarro Castelló	14/05/1984
5	C0006	Isaura Leyre Avilés Pelayo	18/04/1987
6	C0007	Soraya Morera-Lago	27/11/1990
7	C0008	Victoriano Tapia-Cabanillas	06/03/1981
8	C0009	Nidia Saez Campoy	28/04/1980
9	C0010	Teófila Villanueva Molina	04/01/1987
10	C0011	Trini de Alberdi	04/12/1990
11	C0012	Dani Baena	20/10/1977
12	C0013	Angelina de Arregui	21/02/2000
13	C0014	Samuel de Carranza	16/05/1999
14	C0015	Jacinta Montenegro Garcés	13/03/1994
15	C0016	Lisandro Delgado Nadal	18/04/1980

```
json = pd.read_json("datos.json")
print(json)
```

[2 rows x 4 columns]

	Titulo	Tiempo	Final
0	RESPONSIVE WEB DESIGN CON BOOTSTRAP	01:07:20	si
1	Presentación	00:03:20	si
2	*** Definición del proyecto de diseño	NaN	si
3	- Introducción	00:04:50	si
4	- Entorno de trabajo	00:04:50	si
5	- Esquema y estructura del proyecto de diseño	00:07:45	si
6	- Preparación de imágenes en Photoshop	00:06:20	si
7	- Prototipado de la página web	00:05:25	si
8	- Estructura HTML5	00:10:35	si
9	*** Twitter Bootstrap (Framework Responsive De...	NaN	si
10	- Introducción a bootstrap	00:06:25	si
11	- Estructura HTML5 básica para bootstrap	00:04:20	si
12	- Sistema de rejilla (grid system) fijo y fluido	00:11:14	si
13	- Capacidades Responsive (@media - CSS)	00:09:05	si
14	- Base CSS - Tipografías y elementos en línea	00:11:49	si

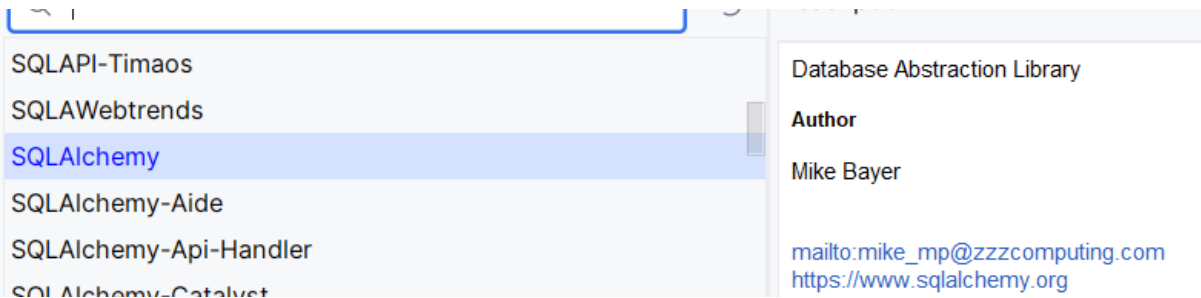
Para trabajar con archivos xml y html necesitamos añadir la librería lxml



```
html = pd.read_html("simple-table.html")
print(html)

[          0  ...          3
0  Hi, I'm your first cell.  ...  I'm your fourth cell.
1  Second row, first cell.  ...          Cell 4.]
```

Para interactuar con bases de datos SQL necesitamos añadir la librería SQLAlchemy



```
import pandas as pd
import sqlite3
import sqlalchemy

#Nos conectamos a la base de datos
con = sqlite3.connect('Our DataBase')
cur = con.cursor()

#Creamos la tabla
cur.execute(''' CREATE TABLE IF NOT EXISTS COURSES
                ([course_id] INTEGER PRIMARY KEY,
                 [course_name] TEXT,
                 [fee] INTEGER,
                 [duration] TEXT,
                 [discount] INTEGER)''')

#Insertamos filas en la tabla
cur.execute('''INSERT INTO COURSES (course_id, course_name, fee, duration, discount)
                VALUES (1,'Spark',25000,'50 Days', 2000),
                (2,'Pandas',20000,'35 Days', 1000),
                (3,'Java',15000,'35 Days', 800),
                (4,'Python',15000,'30 Days', 500),
                (5,'PHP',28000,'30 Days', 800)''')

#Leemos la tabla
sql_query = pd.read_sql('SELECT * FROM COURSES', con)
```

```
print(sql_query)
```

	course_id	course_name	fee	duration	discount
0	1	Spark	25000	50 Days	2000
1	2	Pandas	20000	35 Days	1000
2	3	Java	15000	35 Days	800
3	4	Python	15000	30 Days	500
4	5	PHP	28000	30 Days	800

En el ejemplo de csv vamos a emplear algunos de los argumentos opcionales para ver como funcionarían

```
import pandas as pd
```

```
# Usamos distintos argumentos opcionales
# Especificamos el separador y definimos la fila con los encabezados
lectural1 = pd.read_csv("InventoryLot_v4.csv", sep=";", header=0)
```

```
# Mostramos solo las primeras 5 filas
print(lectural1.head())
```

	product	partNumber	...	storageDate
0	PS-SL-A287	...	2022-01-01T10:00:00	
1	PS-SL-A288	...	2022-01-01T00:00:00	
2	PS-SL-H309	...	2022-01-01T09:00:00	
3	PS-SL-H310	...	2022-01-01T00:30:00	
4	PS-SL-F342	...	2022-01-01T00:00:00	

[5 rows x 15 columns]

```
# Definimos la columna de índices y solo cargamos 3 columnas del fichero
lectura2 = pd.read_csv("InventoryLot_v4.csv", index_col="lotCode",
                        usecols=["lotCode", "quantity", "value"])
```

```
print(lectura2)
```

	quantity	value
lotCode		
521033	25	31250.0
3282509	20	25000.0
862421	3	3750.0
9282139	35	43750.0
240235	43	53750.0
...	...	...
8774537	118	151866.0
9695355	323	470288.0
8012120	273	227409.0
3516489	259	267288.0
4216598	27	26352.0

[154 rows x 2 columns]

```
# Omitimos las primeras 100 líneas y lee solo las siguientes 10
lectura3 = pd.read_csv("InventoryLot_v4.csv", skiprows=100, nrows=10)
print(lectura3)
```

```

      7900014  LT-1  ITEM  387  ...    7353  USD  https://foo.com  2022-01-01T00:00:00
0  7900015  LT-1  ITEM  155  ...     465  USD  https://foo.com  2022-01-01T00:00:00
1  9664144  LT-1  ITEM   37  ...     888  USD  https://foo.com  2022-01-01T00:00:00
2  9665306  LT-1  ITEM  233  ...     699  USD  https://foo.com  2022-01-01T00:00:00
3  2530003  LT-1  ITEM  214  ...  276060  USD  https://foo.com  2022-01-01T00:00:00
4  2530020  LT-1  ITEM  455  ...  536900  USD  https://foo.com  2022-01-01T00:00:00
5  7903986  LT-1  ITEM  228  ...  225720  USD  https://foo.com  2022-01-01T00:00:00
6  8220072  LT-1  ITEM   44  ...   63888  USD  https://foo.com  2022-01-01T00:00:00
7  83008200 LT-1  ITEM  116  ...  278168  USD  https://foo.com  2022-01-01T00:00:00
8  83008300 LT-1  ITEM  109  ...  500528  USD  https://foo.com  2022-01-01T00:00:00
9  83008400 LT-1  ITEM  177  ...  978810  USD  https://foo.com  2022-01-01T00:00:00

```

```
[10 rows x 15 columns]
```

```
# Especificamos que reconozca como valores nulos "N/A", "Desconocido" y vacío.
```

```
# No considera los valores predeterminados de pandas
```

```
lectura4 = pd.read_csv("InventoryLot_v4.csv", na_values=["N/A",
                                                         "Desconocido", ""], keep_default_na=False)
```

```
print(lectura4.info())
```

```
[10 rows x 15 columns]
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 154 entries, 0 to 153
```

```
Data columns (total 15 columns):
```

#	Column	Non-Null Count	Dtype
0	product.partNumber	154 non-null	object
1	location.locationIdentifier	154 non-null	object
2	inventoryType	154 non-null	object
3	quantity	154 non-null	int64
4	quantityUnits	154 non-null	object
5	expirationDate	154 non-null	object
6	inventoryParentType	152 non-null	object
7	class	153 non-null	object
8	segment	151 non-null	object
9	lotCode	154 non-null	int64
10	status	154 non-null	object
11	value	152 non-null	float64
12	valueCurrency	154 non-null	object
13	sourceLink	154 non-null	object
14	storageDate	154 non-null	object

```
dtypes: float64(1), int64(2), object(12)
```

```
memory usage: 18.2+ KB
```

```
None
```

```
# Convertimos la columna expirationDate a formato de fecha
```

```
lectura5 = pd.read_csv("InventoryLot_v4.csv",  
                        parse_dates=["expirationDate"])
```

```
print(lectura5["expirationDate"])
```

```
0      2022-12-31 00:00:00
```

```
1      2022-12-31 09:00:00
```

```
2      2022-12-31 10:00:00
```

```
3      2022-12-31 00:00:00
```

```
4      2022-12-31 05:00:00
```

```
...
```

```
149    2022-12-31 00:00:00
```

```
150    2022-12-31 00:00:00
```

```
151    2022-12-31 00:00:00
```

```
152    2022-12-31 00:00:00
```

```
153    2022-12-31 00:00:00
```

```
Name: expirationDate, Length: 154, dtype: datetime64[ns]
```

EJEMPLOS DE EXPORTACIÓN

```
import pandas as pd

json = pd.read_json("datos.json")
# Exportamos solo las primeras 5 filas
# "records" es el formato que le damos al string json
json.head(5).to_json("exportado.json", orient="records", indent=4)

lectural = pd.read_csv("InventoryLot_v4.csv", sep=";", header=0)

# Exportar todo el dataframe
lectural.to_csv("exportado.csv")

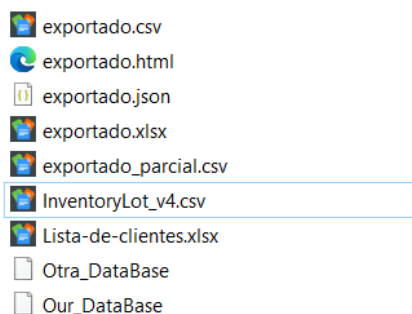
#Exportar algunas columnas y sin que se agregue una columna de índice
lectural[["product.partNumber", "quantity",
          "expirationDate"]].to_csv("exportado_parcial.csv",
                                   index=False)

html = pd.read_html("simple-table.html")
# Exportamos el primer Dataframe (tenemos que especificarlo)
html[0].to_html("exportado.html")

empleados = pd.read_excel("Lista-de-clientes.xlsx")
# Exportamos solo algunas columnas
empleados[["ID", "Nombre completo",
           "Grupo de clientes"]].to_excel("exportado.xlsx",
                                          index=False)

empleados.to_clipboard() #puedes pegarlo en un fichero de texto

con = sqlite3.connect("Otra_DataBase")
# Exportamos el dataframe a SQLite
empleados.to_sql("INVENTORY", con, if_exists="replace",
                index=False)
con.close()
```



Se generarán los distintos ficheros.